# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE CODE: CSA1410** **COURSE NAME: Compiler Design**

---

**COURSE OUTCOME COVERED**

***CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation.*** *(BL3)*

*Assessment Tool Weightage: 40%*

---

**QUESTIONS:4** **Total Marks:50**

Annexure A

## DESIGN TASK

---

***Code of Conduct:***

***I <u>Kola Amrutha Sandhya(192524270)</u>*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***
***Amrutha***

***Course Faculty***
***Dr. N. Deepa***

Annexure A

# DESIGN TASK

## Q1. Predictive Parser Design (Application Level)

**Task:**
*Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.*

.

**Grammar:**
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id

**Conditions:**

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

**Test Case:**
**Input String:**
id + id * id
**Expected Output:**

- Step-by-step parsing actions
- Final result: **String Accepted**

**Q2. Shift-Reduce Parsing (Application Level)**
**Task:**
*Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.*

**Grammar:**
E → E + E
E → E * E
E → id

**Conditions:**

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

**Test Case:**
**Input:**
id + id * id
**Expected Output:**

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

## Q 5. Advanced LL(1) Parser Evaluation and Design

**Task:**
*Analyze and redesign the given grammar to make it suitable for LL(1) parsing, construct the predictive parsing table, and evaluate its correctness by parsing the input string.*

**Grammar:**
S → S + T | T
T → T * F | F
F → (S) | id

**Conditions:**

- Eliminate left recursion completely
- Apply left factoring where necessary
- Compute FIRST and FOLLOW sets accurately
- Construct the LL(1) parsing table
- Justify whether the grammar satisfies LL(1) conditions
- Trace full parsing steps with stack, input, and action

**Test Case:**
Input: id + id * id
Expected: String Accepted with full trace

**Q7. Operator Precedence Parsing Design and Evaluation**

**Task:**
*Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.*

**Grammar:**
E → E + E | E * E | (E) | id

**Conditions:**

- Define operator precedence relations (<, =, >)
- Construct precedence table
- Handle associativity rules explicitly
- Detect and resolve conflicts
- Show step-by-step parsing actions
- Evaluate correctness of parsing decisions

**Test Case:**
Input: id + id * id

**Q8. Ambiguity Removal and Parsing Strategy Evaluation**

**Task:**
*Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.*

**Grammar:**
E → E - E | E / E | id

**Conditions:**

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

**Test Case:**
Input: id - id / id

**Q5. Advanced LL(1) Parser Evaluation and Design**

To eliminate left recursion, construct an LL(1) grammar, and implement a predictive parser for the input id + id * id.

**Original grammar:**

S → S + T | T

T → T * F | F

F → (S) | id

**Grammar after eliminating left recursion:**

S → T S′

S′ → + T S′ | ε

T → F T′

T′ → * F T′ | ε

F → (S) | id

**FIRST sets:**

FIRST(S) = { (, id };

FIRST(S′) = { +, ε };

FIRST(T) = { (, id };

FIRST(T′) = { *, ε };

FIRST(F) = { (, id }.

**FOLLOW sets:**

FOLLOW(S) = { ), $ };

FOLLOW(S′) = { ), $ };

FOLLOW(T) = { +, ), $ };

FOLLOW(T′) = { +, ), $ };

FOLLOW(F) = { *, +, ), $ }.

**Predictive parsing table (non-empty entries):**

| **Nonterminal** | **Lookahead** | **Production** |
|---|---|---|
| S | id or ( | S → T S′ |
| S′ | + | S′ → + T S′ |
| S′ | ) or $ | S′ → ε |
| T | id or ( | T → F T′ |
| T′ | * | T′ → * F T′ |
| T′ | + or ) or $ | T′ → ε |
| F | id | F → id |
| F | ( | F → (S) |

**Algorithm:**

1. Tokenize the input into id, operators, and parentheses, then append $.
2. Initialize the stack with $ and S.
3. If the stack top is a terminal, match it with the next input token.
4. If the stack top is a nonterminal, select the production from the LL(1) table using the next input token.
5. Accept if both the stack and input contain only $; otherwise report an error.

## SOURCE CODE

```
import re
table = {
    ("S", "id"): ["T", "Sp"], ("S", "("): ["T", "Sp"],
    ("Sp", "+"): ["+", "T", "Sp"],
    ("Sp", ")"): [], ("Sp", "$"): [],
    ("T", "id"): ["F", "Tp"], ("T", "("): ["F", "Tp"],
    ("Tp", "*"): ["*", "F", "Tp"],
    ("Tp", "+"): [], ("Tp", ")"): [], ("Tp", "$"): [],
    ("F", "id"): ["id"],
    ("F", "("): ["(", "S", ")"]
}
def tokenize(text):
    tokens = re.findall(r'id|[()+*]', text.replace(" ", ""))
    if "".join(tokens) != text.replace(" ", ""):
        raise ValueError("Invalid token in input")
    return tokens + ["$"]
def parse(text):
    tokens = tokenize(text)
    stack = ["$", "S"]
    i = 0
    print(f'{"Stack":<25} {"Input":<25} Action')
    while stack:
        top = stack[-1]
        lookahead = tokens[i]
        print(f'{" ".join(stack):<25} {" ".join(tokens[i:]):<25}', end=" ")
        if top == "$" and lookahead == "$":
            print("ACCEPT")
            return
        if top in ["id", "+", "*", "(", ")", "$"]:
```

```
        if top == lookahead:
            stack.pop()
            i += 1
            print("Match", top)
        else:
            print("ERROR: terminal mismatch")
            return
    else:
        production = table.get((top, lookahead))
        if production is None:
            print("ERROR: no table entry")
            return
        stack.pop()
        for symbol in reversed(production):
            stack.append(symbol)
        print(f'{top} -> {" ".join(production) if production else "epsilon"}')
try:
    parse("id + id * id")
except ValueError as e:
    print(e)
```

## Sample Output:



```
import re

table = {
    ("S", "id"): ["T", "Sp"], ("S", "("): ["T", "Sp"],
    ("Sp", "+"): ["+", "T", "Sp"],
    ("Sp", ")"): [], ("Sp", "$"): [],
    ("T", "id"): ["F", "Tp"], ("T", "("): ["F", "Tp"],
    ("Tp", "*"): ["*", "F", "Tp"],
    ("Tp", "+"): [], ("Tp", ")"): [], ("Tp", "$"): [],
    ("F", "id"): ["id"],
    ("F", "("): ["(", "S", ")"]
}

def tokenize(text):
    tokens = re.findall(r'id|[()+*]', text.replace(" ", ""))
    if "".join(tokens) != text.replace(" ", ""):
        raise ValueError("Invalid token in input")
    return tokens + ["$"]

def parse(text):
```

| Stack | Input | Action |
|---|---|---|
| $ S | id + id * id $ | S -> T Sp |
| $ Sp T | id + id * id $ | T -> F Tp |
| $ Sp Tp F | id + id * id $ | F -> id |
| $ Sp Tp id | id + id * id $ | Match id |
| $ Sp Tp | + id * id $ | Tp -> epsilon |
| $ Sp | + id * id $ | Sp -> + T Sp |
| $ Sp T + | + id * id $ | Match + |
| $ Sp T | id * id $ | T -> F Tp |
| $ Sp Tp F | id * id $ | F -> id |
| $ Sp Tp id | id * id $ | Match id |
| $ Sp Tp | * id $ | Tp -> * F Tp |
| $ Sp Tp F * | * id $ | Match * |
| $ Sp Tp F | id $ | F -> id |
| $ Sp Tp id | id $ | Match id |
| $ Sp Tp | $ | Tp -> epsilon |
| $ Sp | $ | Sp -> epsilon |
| $ | $ | ACCEPT |

**Result:** Thus, the transformed grammar is suitable for predictive parsing, and the test expression is accepted.

## Q7. Operator Precedence Parsing Design and Evaluation

To implement precedence-based shift/reduce parsing and verify that multiplication has higher precedence than addition.

**Grammar:** E → E + E | E * E | (E) | id

Precedence and associativity rules: * has higher precedence than +. Both + and * are left-associative. Parentheses have the highest grouping priority.

**Simplified precedence guide:**

| Operator | Precedence | Associativity |
|---|---|---|
| + | 1 (lower) | Left |
| * | 2 (higher) | Left |
| ( ) | Grouping | Inner expression first |

**Algorithm:**

Read tokens from left to right. Shift operands to the output. For an operator, reduce (pop) any stacked operator with equal or higher precedence before shifting the new operator. At the end, reduce the remaining operators. This demonstrates precedence-driven shift/reduce decisions.

**Source Code:**

```
import re
precedence = {"+": 1, "*": 2}
operators = set(precedence)
def parse(expression):
    tokens = re.findall(r'id|[()+*]', expression.replace(" ", ""))
    if "".join(tokens) != expression.replace(" ", ""):
        print("ERROR: invalid token")
        return
    op_stack = []
    output = []
    print(f'{"Token":<8} {"Operator stack":<20} {"Postfix/output":<25} Action')
    for token in tokens:
        if token == "id":
            output.append(token)
            action = "Shift id"
        elif token == "(":
            op_stack.append(token)
            action = "Shift ("
        elif token == ")":
            while op_stack and op_stack[-1] != "(":
```

```
            op = op_stack.pop()
            output.append(op)
            action = "Reduce/pop " + op
            print(f'{token:<8} {str(op_stack):<20} {" ".join(output):<25} {action}')
        if not op_stack:
            print("ERROR: unmatched )")
            return
        op_stack.pop()
        action = "Match parentheses"
    elif token in operators:
        action = "Shift " + token
        while op_stack and op_stack[-1] in operators and precedence[op_stack[-1]] >= precedence[token]:
            op = op_stack.pop()
            output.append(op)
            print(f'{token:<8} {str(op_stack):<20} {" ".join(output):<25} Reduce/pop ' + op)
        op_stack.append(token)
    else:
        print("ERROR: unexpected token")
        return
    print(f'{token:<8} {str(op_stack):<20} {" ".join(output):<25} {action}')
if "(" in op_stack:
    print("ERROR: unmatched (")
    return
while op_stack:
    op = op_stack.pop()
    output.append(op)
    print(f'{"$":<8} {str(op_stack):<20} {" ".join(output):<25} Reduce/pop ' + op)
depth = 0
for token in output:
    if token == "id":
        depth += 1
    else:
        if depth < 2:
            print("REJECTED: invalid expression")
            return
        depth -= 1
print("Postfix:", " ".join(output))
```

```
    print("ACCEPTED" if depth == 1 else "REJECTED")
parse("id + id * id")
```

**Expected postfix form:**

id id id * +

**Sample Output:**



```
import re
precedence = {"+": 1, "*": 2}
operators = set(precedence)
def parse(expression):
    tokens = re.findall(r'id|[()+*]', expression.replace(" ", ""))
    if "".join(tokens) != expression.replace(" ", ""):
        print("ERROR: invalid token")
        return
    op_stack = []
    output = []
    print(f'{"Token":<8} {"Operator stack":<20} {"Postfix/output":<25} Action')

    for token in tokens:
        if token == "id":
            output.append(token)
            action = "Shift id"
        elif token == "(":
            op_stack.append(token)
```

```
Token    Operator stack       Postfix/output            Action
id       []                   id                        Shift id
+        ['+']                id                        Shift +
id       ['+']                id id                     Shift id
*        ['+', '*']           id id                     Shift *
id       ['+', '*']           id id id                  Shift id
$        ['+']                id id id *                Reduce/pop *
$        []                   id id id * +              Reduce/pop +
Postfix: id id id * +
ACCEPTED
```

## Q8. Ambiguity Removal and Parsing Strategy Evaluation

To remove ambiguity by defining operator precedence and implement a precedence-driven shift/reduce parser for id - id / id.

**Original grammar:** E → E - E | E / E | id

**Unambiguous grammar (division has higher precedence than subtraction; both operators are left-associative):**

E → E - T | T

T → T / F | F

F → id

**Explanation:**

The original grammar is ambiguous because it does not specify whether - or / should be evaluated first. The revised grammar places division in T, making / bind more tightly than -.

**Algorithm:**

Tokenize the expression. Shift operands to output. Before shifting an operator, reduce any stacked operator with equal or higher precedence. Reduce remaining operators and validate the resulting postfix expression.

**Source Code:**

```
import re
precedence = {"-": 1, "/": 2}
operators = set(precedence)
def parse(expression):
    tokens = re.findall(r'id|[()/\-]', expression.replace(" ", ""))
    if "".join(tokens) != expression.replace(" ", ""):
        print("ERROR: invalid token")
        return
    op_stack = []
    output = []
    print(f'{"Token":<8} {"Operator stack":<20} {"Postfix/output":<25} Action')
    for token in tokens:
        if token == "id":
            output.append(token)
            action = "Shift id"
        elif token == "(":
            op_stack.append(token)
            action = "Shift ("
        elif token == ")":
            while op_stack and op_stack[-1] != "(":
                op = op_stack.pop()
```

```
            output.append(op)
            print(f'{token:<8} {str(op_stack):<20} {" ".join(output):<25} Reduce/pop ' + op)
        if not op_stack:
            print("ERROR: unmatched )")
            return
        op_stack.pop()
        action = "Match parentheses"
    elif token in operators:
        action = "Shift " + token
        while op_stack and op_stack[-1] in operators and precedence[op_stack[-1]] >= precedence[token]:
            op = op_stack.pop()
            output.append(op)
            print(f'{token:<8} {str(op_stack):<20} {" ".join(output):<25} Reduce/pop ' + op)
        op_stack.append(token)
    else:
        print("ERROR: unexpected token")
        return
    print(f'{token:<8} {str(op_stack):<20} {" ".join(output):<25} {action}')

if "(" in op_stack:
    print("ERROR: unmatched (")
    return
while op_stack:
    op = op_stack.pop()
    output.append(op)
    print(f'{"$":<8} {str(op_stack):<20} {" ".join(output):<25} Reduce/pop ' + op)
depth = 0
for token in output:
    if token == "id":
        depth += 1
    else:
        if depth < 2:
            print("REJECTED: invalid expression")
            return
        depth -= 1
print("Postfix:", " ".join(output))
print("ACCEPTED" if depth == 1 else "REJECTED")
```

parse("id - id / id")

**Expected postfix form:**

id id id / -

**Sample Output:**



```
import re
precedence = {"-": 1, "/": 2}
operators = set(precedence)
def parse(expression):
    tokens = re.findall(r'id|[()/\-]', expression.replace(" ",
""))
    if "".join(tokens) != expression.replace(" ", ""):
        print("ERROR: invalid token")
        return
    op_stack = []
    output = []
    print(f'{"Token":<8} {"Operator stack":<20}
{"Postfix/output":<25} Action')
    for token in tokens:
        if token == "id":
            output.append(token)
            action = "Shift id"
        elif token == "(":
            op_stack.append(token)
            action = "Shift ("
```

```
Token    Operator stack       Postfix/output            Action
id       []                   id                        Shift id
-        ['-']                id                        Shift -
id       ['-']                id id                     Shift id
/        ['-', '/']           id id                     Shift /
id       ['-', '/']           id id id                  Shift id
$        ['-']                id id id /                Reduce/pop /
$        []                   id id id / -              Reduce/pop -
Postfix: id id id / -
ACCEPTED
```

**Result:** The revised grammar and precedence rules resolve the ambiguity for the specified operators.

CO2_AT1 - Design Task
compiler Design

K. Amrutha Sandhya
192524270.

1, Predictive parser Design (LL(1))

To construct an LL(1) predictive parsing table and Parse the input string id+ id* id.

Given Grammar

$E \rightarrow TE'$ $\quad F \rightarrow (E) \mid id$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

step2:-

FIRST()

FIRST(E) = { (, id}

FIRST(E') = { +, ε}

FIRST(T) = { (, id}

FIRST(T') = {*, ε}

FIRST(F) = { (, id}

FOLLOW()

FOLLOW(E) = {$, )}

FOLLOW(E') = {$, )}

FOLLOW(T) = {+, $, )}

FOLLOW(T') = {+, $, )}

FOLLOW(F) = {*, +, ), $}

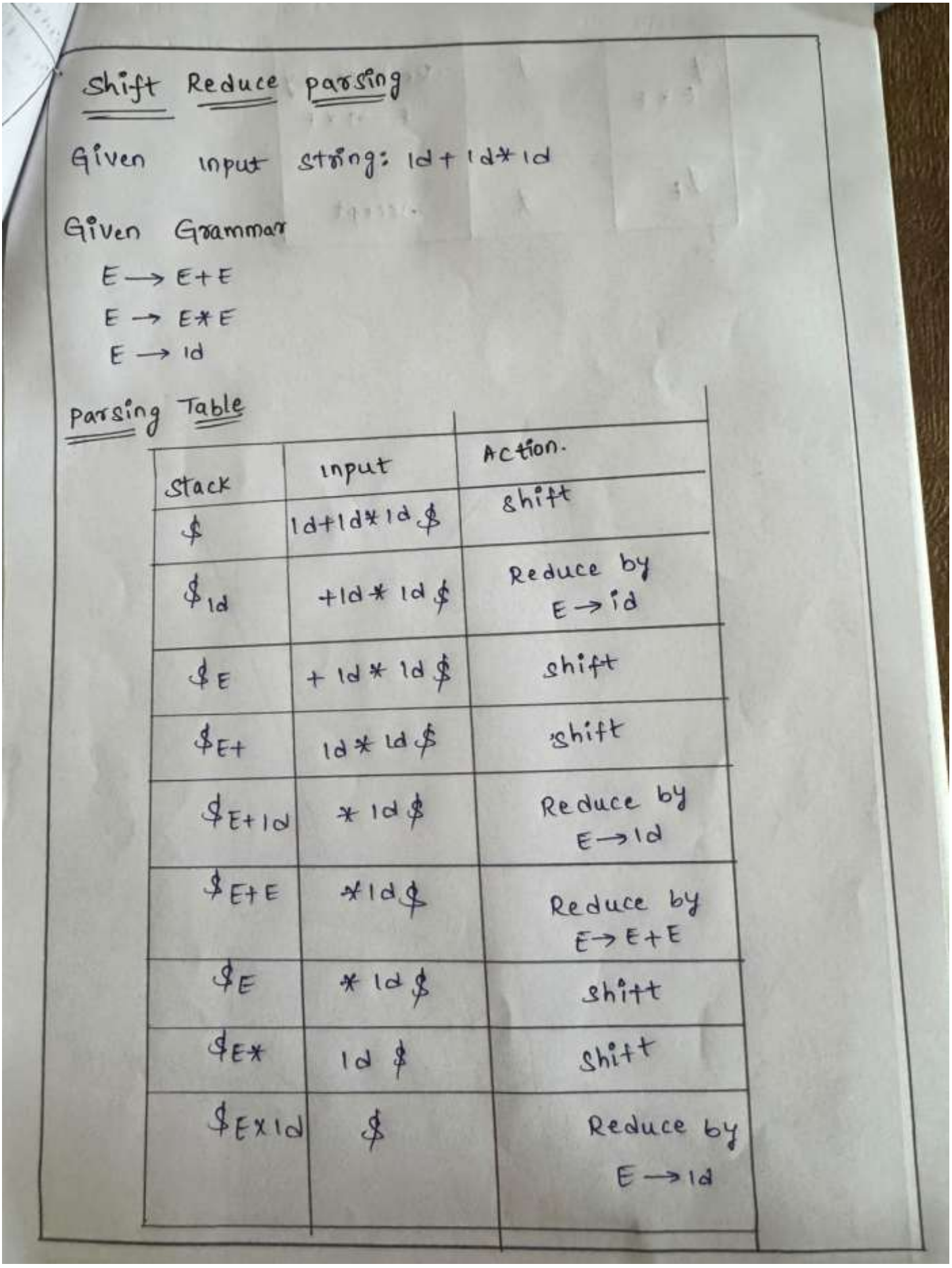
Step 3:- predictive parsing Table

| Non Terminal | + | * | ( | ) | $ | id |
|---|---|---|---|---|---|---|
| E | | | $E \to TE'$ | | | $E \to TE'$ |
| $E'$ | $E' \to +TE'$ | | | $E' \to \varepsilon$ | $E' \to \varepsilon$ | |
| T | | | $T \to FT'$ | | | $T \to FT'$ |
| $T'$ | $T' \to \varepsilon$ | $T' \to *FT'$ | | $T' \to \varepsilon$ | $T' \to \varepsilon$ | |
| F | | | | $F \to (E)$ | | $F \to id$ |

Stack implementation:-

| Stack | Input | output |
|---|---|---|
| $\$E$ | id+id*id $ | $E \to TE'$ |
| $\$E'T$ | id+id*id $ | $T \to FT'$ |
| $\$E'T'F$ | id+id*id $ | $F \to id$ |
| $\$E'T'id$ | id+id*id $ | $T' \to \varepsilon$ |
| $\$E'$ | +id*id $ | $E' \to +TE'$ |
| $\$E'T+$ | +id*id $ | $T \to FT'$ |
| $\$E'T'F$ | id*id $ | $F \to id$ |
| $\$E'T'id$ | id*id $ | $TE' \to *FT'$ |
| $\$E'T'F*$ | *id $ | $FE' \to id$ |
| $\$E'T'id$ | id $ | $T' \to \varepsilon$ |
| $\$E'$ | $ | $E' \to \varepsilon$ |
| $ | $ | |

**Q2. Shift-Reduce Parsing (Application Level)**



Shift Reduce parsing

Given input string: id + id * id

Given Grammar

$E \rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow id$

Parsing Table

| Stack | input | Action. |
|---|---|---|
| $ | id+id*id $ | shift |
| $id | +id * id $ | Reduce by E → id |
| $E | + id * id $ | shift |
| $E+ | id * id $ | shift |
| $E+id | * id $ | Reduce by E → id |
| $E+E | *id $ | Reduce by E → E+E |
| $E | * id $ | shift |
| $E* | id $ | shift |
| $E*id | $ | Reduce by E → id |

| | | |
|---|---|---|
| $E*E | $ | Reduce by E → E*E |
| $E | $ | Accept |

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 20 | Demonstrates thorough understanding and integrates multiple concepts effectively | Good understanding with proper integration of most concepts | Basic understanding with partial integration | Poor understanding with no integration |
| Design & Implementation | 25 | Well-designed solution with systematic and optimal implementation | Correct design with minor issues | Basic design with errors or inefficiencies | Incorrect or incomplete design |
| Parser/Algorithm Construction | 20 | Accurate construction of parser/algorithm with complete logic | Mostly correct construction with minor errors | Partial construction with missing logic | Incorrect or incomplete construction |
| Analysis & Validation | 15 | Insightful analysis with correct validation and justification | Good analysis with reasonable justification | Basic analysis with limited validation | Poor or incorrect analysis |
| Documentation & Presentation | 20 | Well-structured documentation with diagrams, steps, and clarity | Organized documentation with required details | Basic documentation with missing details | Poor or incomplete documentation |